

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANSWER KEY

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME (CO3): Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

Activity Tool: Experiments (High)

Assessment Tool Weightage: 40%

Question 1 (5 Marks)

For the relation *STUDENT_COURSE*(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Step 1 — Identify the functional dependencies (FDs)

Reasoning about the real-world meaning of each attribute gives the following FDs:

- $SID \rightarrow SName$ (each student id has exactly one student name)
- $CID \rightarrow CName, Instructor$ (each course id has exactly one course name and one instructor)
- $SID, CID \rightarrow Grade$ (a grade is awarded only for a specific student in a specific course)

Step 2 — Compute the candidate key using attribute closure

Only {SID, CID} together determine every other attribute, so test its closure:

Closure step	Attributes derived	Running closure (SID,CID)+
Start	SID, CID	{SID, CID}
Apply $SID \rightarrow SName$	SName	{SID, CID, SName}
Apply $CID \rightarrow CName, Instructor$	CName, Instructor	{SID, CID, SName, CName, Instructor}
Apply $SID, CID \rightarrow Grade$	Grade	{SID, CID, SName, CName, Instructor, Grade} = all attributes

Since (SID,CID)+ equals the full attribute set, {SID, CID} is a super key. Neither {SID} nor {CID} alone reaches every attribute (SID+ = {SID, SName} only; CID+ = {CID, CName, Instructor} only), so no proper subset is a key.

SQL demonstration

```
CREATE TABLE STUDENT_COURSE(  
  SID INTEGER, SName TEXT, CID TEXT, CName TEXT,  
  Instructor TEXT, Grade TEXT,  
  PRIMARY KEY (SID, CID)  
);  
  
INSERT INTO STUDENT_COURSE VALUES  
(101, 'Arun', 'C101', 'DBMS', 'Dr. Mehta', 'A'),  
(101, 'Arun', 'C102', 'OS', 'Dr. Rao', 'B'),  
(102, 'Divya', 'C101', 'DBMS', 'Dr. Mehta', 'A'),  
(103, 'Kiran', 'C103', 'CN', 'Dr. Iyer', 'B+');
```

```
SELECT * FROM STUDENT_COURSE;
```

Output:

SID	SName	CID	CName	Instructor	Grade
101	Arun	C101	DBMS	Dr. Mehta	A
101	Arun	C102	OS	Dr. Rao	B
102	Divya	C101	DBMS	Dr. Mehta	A
103	Kiran	C103	CN	Dr. Iyer	B+

Conclusion: The candidate key of STUDENT_COURSE is {SID, CID}. The FDs $SID \rightarrow SName$ and $CID \rightarrow (CName, Instructor)$ are partial dependencies on this composite key, so the relation is not in 2NF — this is resolved in Question 3's style of decomposition.

Question 2 (5 Marks)

Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Given (unnormalized) relation

ORDERS_UNF(OrderID, Customer, Products) — the Products attribute stores a comma-separated, multi-valued list inside a single cell, which violates the atomicity rule of 1NF (every attribute must hold a single, indivisible value).

```
CREATE TABLE ORDERS_UNF(OrderID INT, Customer TEXT, Products TEXT);
INSERT INTO ORDERS_UNF VALUES
  (1, 'Rahul', 'Pen,Notebook,Eraser'),
  (2, 'Sneha', 'Pencil,Ruler');
SELECT * FROM ORDERS_UNF;
```

Output:

OrderID	Customer	Products
1	Rahul	Pen,Notebook,Eraser
2	Sneha	Pencil,Ruler

Step-by-step conversion to 1NF

- Identify the repeating/multi-valued group: Products.
- Remove the repeating group by giving each value its own row, while the atomic attributes (OrderID, Customer) repeat alongside it.
- Redefine the primary key as the composite (OrderID, Product) since OrderID alone no longer uniquely identifies a row.

```
CREATE TABLE ORDERS_1NF(OrderID INT, Customer TEXT, Product TEXT,
                        PRIMARY KEY(OrderID, Product));
INSERT INTO ORDERS_1NF VALUES
  (1, 'Rahul', 'Pen'), (1, 'Rahul', 'Notebook'), (1, 'Rahul', 'Eraser'),
  (2, 'Sneha', 'Pencil'), (2, 'Sneha', 'Ruler');
SELECT * FROM ORDERS_1NF;
```

Output:

OrderID	Customer	Product
1	Rahul	Pen
1	Rahul	Notebook
1	Rahul	Eraser
2	Sneha	Pencil
2	Sneha	Ruler

Conclusion: Every cell now holds a single atomic value and every row is uniquely identified by (OrderID, Product), so ORDERS_1NF satisfies 1NF. (Note: Customer still repeats per order — a further 2NF/3NF split into ORDERS(OrderID, Customer) and ORDER_ITEMS(OrderID, Product) would remove that redundancy, as shown in later questions.)

Question 3 (5 Marks)

Given relation $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Step 1 — Find the candidate key

Compute A^+ :

Step	Rule applied	Closure so far
1	Start	$\{A\}$
2	$A \rightarrow BC$	$\{A,B,C\}$
3	$BC \rightarrow D$	$\{A,B,C,D\}$
4	$D \rightarrow E$	$\{A,B,C,D,E\} = R$

$A^+ =$ all attributes of R , so $\{A\}$ alone is a candidate key. Because the key is a single attribute, no proper non-empty subset of it exists — so, strictly, R cannot contain a partial dependency and is already in 2NF with respect to key $\{A\}$.

Step 2 — Why decomposition is still needed

Although 2NF is technically satisfied, the FD chain $A \rightarrow BC \rightarrow D \rightarrow E$ creates transitive dependencies ($A \rightarrow D$ and $A \rightarrow E$ hold only via B,C and D respectively), which cause update/insertion redundancy. Applying the same synthesis technique used for eliminating partial/transitive dependencies, R is decomposed along each FD:

Decomposed relation	Key	FD represented
$R1(A,B,C)$	A	$A \rightarrow BC$
$R2(B,C,D)$	B,C	$BC \rightarrow D$
$R3(D,E)$	D	$D \rightarrow E$

SQL demonstration and lossless-join check

```
CREATE TABLE R1(A INT PRIMARY KEY, B TEXT, C TEXT);
INSERT INTO R1 VALUES (1, 'B1', 'C1'), (2, 'B2', 'C2');

CREATE TABLE R2(B TEXT, C TEXT, D TEXT, PRIMARY KEY(B,C));
INSERT INTO R2 VALUES ('B1', 'C1', 'D1'), ('B2', 'C2', 'D2');

CREATE TABLE R3(D TEXT PRIMARY KEY, E TEXT);
INSERT INTO R3 VALUES ('D1', 'E1'), ('D2', 'E2');

SELECT r1.A, r2.B, r2.C, r2.D, r3.E
FROM R1 r1 JOIN R2 r2 ON r1.B=r2.B AND r1.C=r2.C
JOIN R3 r3 ON r2.D=r3.D;
```

Output:

A	B	C	D	E
1	B1	C1	D1	E1
2	B2	C2	D2	E2

Conclusion: The natural join of $R1$, $R2$, $R3$ reconstructs R exactly (no lost or spurious rows), confirming a lossless-join decomposition that removes the redundancy caused by the FD chain, while remaining fully in 2NF (in fact 3NF).

Question 4 (5 Marks)

Apply 3NF decomposition on the relation $EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)$ with transitive dependencies.

Step 1 — Functional dependencies

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$ (key FD)
- $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$ (non-key attribute determining other non-key attributes)

Candidate key: {EmpID}. The FD $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$ is a transitive dependency because $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}/\text{ManagerID}$, i.e. non-prime attributes depend on the key only through another non-prime attribute (DeptID). This violates 3NF.

Step 2 — 3NF decomposition

Relation	Attributes	Key
EMPLOYEE	EmpID, EmpName, DeptID	EmpID
DEPARTMENT	DeptID, DeptName, ManagerID	DeptID

```
CREATE TABLE DEPARTMENT(DeptID INT PRIMARY KEY, DeptName TEXT, ManagerID INT);
INSERT INTO DEPARTMENT VALUES (10, 'Sales', 101), (20, 'IT', 102);
```

```
CREATE TABLE EMPLOYEE(EmpID INT PRIMARY KEY, EmpName TEXT,
                        DeptID INT REFERENCES DEPARTMENT(DeptID));
INSERT INTO EMPLOYEE VALUES (1, 'Vikram', 10), (2, 'Anita', 10), (3, 'Suresh', 20);
```

```
SELECT e.EmpID, e.EmpName, e.DeptID, d.DeptName, d.ManagerID
FROM EMPLOYEE e JOIN DEPARTMENT d ON e.DeptID = d.DeptID;
```

Output:

EmpID	EmpName	DeptID	DeptName	ManagerID
1	Vikram	10	Sales	101
2	Anita	10	Sales	101
3	Suresh	20	IT	102

Conclusion: DeptName and ManagerID are now stored once per department instead of once per employee, removing update anomalies (e.g. renaming a department no longer requires editing every employee row) while the join still reproduces the original data.

Question 5 (5 Marks)

Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Step 1 — Find all candidate keys

Attribute set tested	Closure	Full closure (= ABCD)?
AB	$AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A \Rightarrow \{A,B,C,D\}$	Yes — candidate key
BC	$C \rightarrow D$, $D \rightarrow A \Rightarrow \{A,B,C,D\}$	Yes — candidate key
BD	$D \rightarrow A$, then $AB \rightarrow C$ (A,B present) $\Rightarrow \{A,B,C,D\}$	Yes — candidate key
C alone	$C \rightarrow D$, $D \rightarrow A \Rightarrow \{A,C,D\}$	No (missing B)
D alone	$D \rightarrow A \Rightarrow \{A,D\}$	No (missing B,C)

Candidate keys = {AB, BC, BD}. Every attribute (A, B, C, D) appears in at least one candidate key, so all attributes are prime — R therefore already satisfies 3NF. BCNF is stricter, however: it requires every determinant to be a super key.

Step 2 — BCNF test on each FD

FD	Is the determinant a super key?	BCNF holds for this FD?
$AB \rightarrow C$	AB is a candidate key — yes	OK
$C \rightarrow D$	$C^+ = \{A,C,D\}$, missing B — no	Violates BCNF
$D \rightarrow A$	$D^+ = \{A,D\}$, missing B,C — no	Violates BCNF

Step 3 — BCNF decomposition algorithm

- $C \rightarrow D$ violates BCNF. Split R using $X=C, Y=D$: $R_1(C,D)$ and $R_2 = R - D = (A,B,C)$.
- Within $R_2(A,B,C)$, the dependency $C \rightarrow A$ still holds (derived transitively from $C \rightarrow D, D \rightarrow A$) and C is not a super key of R_2 (C^+ within $R_2 = \{A,C\}$, missing B) — R_2 still violates BCNF.
- Split R_2 using $X=C, Y=A$: $R_3(A,C)$ and $R_4 = R_2 - A = (B,C)$.

Final BCNF decomposition: $R_1(C, D)$, $R_3(A, C)$, $R_4(B, C)$ — each relation is trivially in BCNF (in R_1 and R_3 , C is a super key; in R_4 , $\{B,C\}$ is the whole relation, so it is all-key).

```
CREATE TABLE CD_TBL(C TEXT PRIMARY KEY, D TEXT);
INSERT INTO CD_TBL VALUES ('C1','D1'),('C2','D2');

CREATE TABLE AC_TBL(C TEXT PRIMARY KEY, A TEXT);
INSERT INTO AC_TBL VALUES ('C1','A1'),('C2','A2');

CREATE TABLE BC_TBL(B TEXT, C TEXT, PRIMARY KEY(B,C));
INSERT INTO BC_TBL VALUES ('B1','C1'),('B2','C2');

-- Lossless join reconstruction of R(A,B,C,D)
SELECT bc.B, ac.C, ac.A, cd.D
FROM BC_TBL bc JOIN AC_TBL ac ON bc.C=ac.C
JOIN CD_TBL cd ON bc.C=cd.C;
```

Output:

```
B  C  A  D
B1 C1 A1 D1
B2 C2 A2 D2
```

Conclusion: $R(A,B,C,D)$ is NOT in BCNF because $C \rightarrow D$ and $D \rightarrow A$ have non-super-key determinants. Decomposing into $R_1(C,D)$, $R_3(A,C)$, $R_4(B,C)$ removes both violations and the 3-way natural join is lossless.

Question 6 (5 Marks)

Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Relation and decomposition under test

Let $R(A,B,C,D)$ with FD set $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$, decomposed into $R_1(A,B)$, $R_2(B,C)$, $R_3(C,D)$. The Chase algorithm builds a tableau with one row per decomposed relation, filling matching columns with the relation's attribute name (unsubscripted symbol, meaning 'shared') and non-matching columns with subscripted symbols (b_{ij} meaning 'row i, column j, unknown value'), then repeatedly applies each FD in F to equate symbols. If any row becomes all-unsubscripted (i.e. equal to a,b,c,d), the decomposition is lossless.

Initial tableau

Row (relation)	A	B	C	D
t1 ($R_1: A,B$)	a	b	c ₁₃	d ₁₄
t2 ($R_2: B,C$)	a ₂₁	b	c	d ₂₄
t3 ($R_3: C,D$)	a ₃₁	b ₃₂	c	d

Applying the FDs

- $A \rightarrow B$: t1 and t2 already agree on A? No — t1.A=a, t2.A=a21 (different). $A \rightarrow B$ only fires when the A-values agree, so it does not yet apply between t1/t2.
- $B \rightarrow C$: t1.B = b and t2.B = b (both unsubscripted 'b'), so $B \rightarrow C$ forces t1.C = t2.C. Since t2.C = c (unsubscripted), set t1.C = c (replacing c13 with c).
- $C \rightarrow D$: now t1.C = c and t2.C = c and t3.C = c — all three agree, so $C \rightarrow D$ forces all their D-values equal. t3.D = d is unsubscripted, so set t1.D = d and t2.D = d.

Resulting (chased) tableau

Row	A	B	C	D
t1	a	b	c	d
t2	a21	b	c	d
t3	a31	b32	c	d

Conclusion: Row t1 became fully unsubscripted (a, b, c, d), matching the target row of R exactly. By the Chase theorem, this proves the decomposition $\{R1(A,B), R2(B,C), R3(C,D)\}$ is a lossless-join decomposition with respect to F.

Question 7 (5 Marks)

Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Step 1 — Identify the multi-valued dependencies (MVDs)

A teacher can teach several subjects and independently pursue several hobbies; a subject has no bearing on which hobbies that teacher has, and vice-versa. This independence between two multi-valued facts about the same key attribute is exactly the definition of an MVD:

- $TID \twoheadrightarrow Subject$
- $TID \twoheadrightarrow Hobby$

Because both Subject and Hobby vary independently for a fixed TID, storing them together forces every (Subject, Hobby) combination to be repeated for each teacher — this is the classic 4NF anomaly (no FDs are violated; TEACHER can already be in BCNF, yet redundancy remains).

Step 2 — Decompose into 4NF

The 4NF decomposition rule splits R on an MVD $X \twoheadrightarrow Y$ into $R1(X,Y)$ and $R2(X, R-X-Y)$:

```
CREATE TABLE TEACHER_SUBJECT(TID INT, Subject TEXT, PRIMARY KEY(TID,Subject));
INSERT INTO TEACHER_SUBJECT VALUES (1,'Maths'),(1,'Physics'),(2,'Chemistry');

CREATE TABLE TEACHER_HOBBY(TID INT, Hobby TEXT, PRIMARY KEY(TID,Hobby));
INSERT INTO TEACHER_HOBBY VALUES (1,'Chess'),(1,'Painting'),(2,'Cricket');

SELECT * FROM TEACHER_SUBJECT;
SELECT * FROM TEACHER_HOBBY;
```

Output:

```
TID Subject
1 Maths
1 Physics
2 Chemistry
TID Hobby
1 Chess
1 Painting
2 Cricket
```

Conclusion: TEACHER_SUBJECT and TEACHER_HOBBY are each in 4NF: every remaining MVD is trivial. Instead of 2 subjects × 2 hobbies = 4 redundant rows for teacher 1 in the original design, only 2+2 = 4 non-redundant facts are stored, and no combination is duplicated.

Question 8 (5 Marks)

Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Classic example: SUPPLY(Supplier, Part, Project)

Business rule: 'if supplier S supplies part P, and part P is used in project J, and supplier S supplies project J, then S supplies part P specifically for project J.' This circular business rule is a join dependency (JD): $\{ \{ (Supplier, Part), (Part, Project), (Supplier, Project) \} \}$ — the relation cannot be losslessly decomposed into any 2 of these projections; all 3 are required together.

```
CREATE TABLE SPJ(Supplier TEXT, Part TEXT, Project TEXT);
INSERT INTO SPJ VALUES
  ('S1', 'P1', 'J1'), ('S1', 'P2', 'J1'), ('S2', 'P1', 'J1'), ('S1', 'P1', 'J2');
SELECT * FROM SPJ; -- combined relation: redundant
```

Output:

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S2	P1	J1
S1	P1	J2

Notice ('S1','P1') repeats across two rows purely because S1 has multiple parts and projects — this is redundancy caused by the join dependency, not by any FD or MVD, so it is only removable at the 5NF level.

Decompose into three binary projections

```
CREATE TABLE SP(Supplier TEXT, Part TEXT, PRIMARY KEY(Supplier,Part));
INSERT INTO SP SELECT DISTINCT Supplier,Part FROM SPJ;

CREATE TABLE PJ(Part TEXT, Project TEXT, PRIMARY KEY(Part,Project));
INSERT INTO PJ SELECT DISTINCT Part,Project FROM SPJ;

CREATE TABLE SJ(Supplier TEXT, Project TEXT, PRIMARY KEY(Supplier,Project));
INSERT INTO SJ SELECT DISTINCT Supplier,Project FROM SPJ;

-- verify: 3-way natural join must reproduce SPJ exactly
SELECT sp.Supplier, sp.Part, pj.Project
FROM SP sp JOIN PJ pj ON sp.Part=pj.Part
JOIN SJ sj ON sp.Supplier=sj.Supplier AND pj.Project=sj.Project;
```

Output:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S2	P1	J1

Conclusion: The 3-way join of SP, PJ and SJ reproduces the original 4 rows exactly — no more, no fewer — confirming a lossless-join, dependency-preserving 5NF decomposition that eliminates the JD-based redundancy.

Question 9 (5 Marks)

Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Given schema

COLLEGE(SID, SName, DeptID, DeptName, HODName) — a single flat table storing student and department details together.

```
CREATE TABLE COLLEGE_UNNORM(SID INT, SName TEXT, DeptID INT, DeptName TEXT, HODName TEXT);
INSERT INTO COLLEGE_UNNORM VALUES
  (1, 'Naveen', 5, 'CSE', 'Dr. Kumar'),
  (2, 'Priya', 5, 'CSE', 'Dr. Kumar'),
  (3, 'Ravi', 6, 'ECE', 'Dr. Suri');
SELECT * FROM COLLEGE_UNNORM;
```

Output:

SID	SName	DeptID	DeptName	HODName
1	Naveen	5	CSE	Dr. Kumar
2	Priya	5	CSE	Dr. Kumar
3	Ravi	6	ECE	Dr. Suri

Anomalies identified

- Insertion anomaly: a newly opened department (e.g. 'AI&DS') cannot be recorded until at least one student enrolls in it, because DeptName/HODName only exist attached to a student row.
- Deletion anomaly: if Ravi (the only ECE student) is removed, all information about the ECE department and its HOD (Dr. Suri) is lost as a side effect.
- Update anomaly: if Dr. Kumar is replaced as HOD of CSE, every row where DeptID=5 (Naveen's and Priya's) must be updated; missing even one row leaves the database inconsistent.

Normalization to 3NF

FDs: SID → SName, DeptID and DeptID → DeptName, HODName. DeptID→(DeptName,HODName) is a transitive dependency on the key SID, so decompose:

```
CREATE TABLE DEPARTMENT(DeptID INT PRIMARY KEY, DeptName TEXT, HODName TEXT);
INSERT INTO DEPARTMENT VALUES (5, 'CSE', 'Dr. Kumar'), (6, 'ECE', 'Dr. Suri');

CREATE TABLE STUDENT(SID INT PRIMARY KEY, SName TEXT,
  DeptID INT REFERENCES DEPARTMENT(DeptID));
INSERT INTO STUDENT VALUES (1, 'Naveen', 5), (2, 'Priya', 5), (3, 'Ravi', 6);

SELECT * FROM STUDENT;
SELECT * FROM DEPARTMENT;
```

Output:

SID	SName	DeptID
1	Naveen	5
2	Priya	5
3	Ravi	6

DeptID	DeptName	HODName
5	CSE	Dr. Kumar
6	ECE	Dr. Suri

Conclusion: All three anomalies are resolved: a department can now exist in DEPARTMENT with zero students, deleting a student no longer deletes department facts, and the HOD name is updated in exactly one row.

Question 10 (5 Marks)

Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Given relation and FD set

$R(A, B, C, D, E, F)$ with $F = \{ A \rightarrow B, A \rightarrow C, CD \rightarrow E, CD \rightarrow F, B \rightarrow D \}$

General algorithm for attribute-set closure X^+

- Initialize result = X.
- Repeat: for every FD ($Y \rightarrow Z$) in F, if $Y \subseteq \text{result}$, add Z to result.
- Stop when no more attributes can be added.

Closure of $\{A\}$

Iteration	FD applied	Result
0	—	$\{A\}$
1	$A \rightarrow B$	$\{A, B\}$
2	$A \rightarrow C$	$\{A, B, C\}$
3	$B \rightarrow D$	$\{A, B, C, D\}$
4	$CD \rightarrow E, CD \rightarrow F$	$\{A, B, C, D, E, F\} = R$

$A^+ = R$, so $\{A\}$ is a candidate key on its own — no smaller subset ($\emptyset$) can be a key, so $\{A\}$ is minimal.

Checking other single attributes and pairs (to confirm no other minimal key exists)

Set	Closure	Is a key?
B^+	$\{B, D\}$	No — missing A, C, E, F
C^+	$\{C\}$	No
D^+	$\{D\}$	No
CD^+	$\{C, D, E, F\}$	No — missing A, B
BC^+	$B \rightarrow D, C \text{ alone; then } CD \rightarrow E, F \Rightarrow \{B, C, D, E, F\}$	No — missing A

Conclusion: Since every attribute set that excludes A fails to reach the full relation, and A alone already does, $\{A\}$ is the unique candidate key of R. This closure-based method generalizes to any FD set: compute X^+ for every candidate subset (starting from single attributes) and keep only the minimal sets whose closure is the full relation.